

DAT62-2

Neural Networks

A pure implementation course

From the Math S4 formulation to working PyTorch: networks, CNNs, transfer learning,
and the Transformer.

Albert School — 22 hours

Contents

1	From math to code: a feedforward network in PyTorch	2
1.1	Tensors and autograd	2
1.2	Defining the network	2
1.3	The training loop	2
2	Convolutional neural networks	3
2.1	Convolutional layers	3
2.2	Pooling	3
2.3	Receptive fields	3
3	Transfer learning	3
4	Training diagnostics	4
4.1	Reading loss and accuracy curves	4
4.2	Remediation	4
5	The Transformer as a building block	4
5.1	Encoder and decoder stacks	4
5.2	Positional encoding	4
5.3	The full forward path	5
6	Model comparison methodology	5
7	End-to-end neural network project	5

1 From math to code: a feedforward network in PyTorch

This course assumes the mathematics of neural networks from Math S4 — the forward pass, backpropagation, gradient flow, attention, and regularization — and the optimizers from Math S6 (gradient descent, SGD, Adam). It does not re-derive them; it translates them into code.

1.1 Tensors and autograd

A **tensor** is PyTorch’s multi-dimensional array, the data type holding inputs, parameters, and activations. A tensor created with `requires_grad=True` records the operations applied to it, so PyTorch can compute derivatives automatically.

Autograd is this automatic differentiation engine. The forward pass builds a computational graph; calling `.backward()` on a scalar loss applies the chain rule through that graph and stores, in each parameter’s `.grad`, the gradient of the loss with respect to that parameter. This is exactly the backpropagation of Math S4, performed by the library rather than by hand.

1.2 Defining the network

A feedforward network is the matrix formulation of Math S4 written as tensor operations. Each layer computes $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$ followed by a nonlinear activation; in PyTorch a linear layer is `nn.Linear(in, out)` and an activation such as ReLU is `nn.ReLU()`. A model subclasses `nn.Module` and implements a `forward` method that chains these layers from input to output. The parameters $\mathbf{W}$ and $\mathbf{b}$ of every layer are created and tracked automatically.

1.3 The training loop

Training repeats one loop over mini-batches of the dataset. Each iteration:

1. runs the **forward pass** to compute predictions;
2. evaluates the **loss function** comparing predictions to targets (e.g. `nn.CrossEntropyLoss` for classification, `nn.MSELoss` for regression);
3. calls `optimizer.zero_grad()` to clear old gradients;

4. calls `loss.backward()` to compute new gradients by autograd;
5. calls `optimizer.step()` to update the parameters.

The **optimizer** (e.g. `torch.optim.SGD` or `torch.optim.Adam`) implements the update rules from Math S6. One full pass over the dataset is an **epoch**; training runs for several epochs on a real dataset.

2 Convolutional neural networks

A CNN is the one architecture the Math course does not cover. It is designed for data with **spatial structure**, such as images, where nearby pixels are related and a pattern may appear anywhere in the image.

2.1 Convolutional layers

A **convolutional layer** slides a small array of weights, a **kernel** (or filter), across the input and computes, at each position, the sum of element-wise products between the kernel and the local patch it covers. The result is a **feature map**. Because the same kernel is reused at every position, the layer has few parameters and detects the same local pattern wherever it occurs. A layer applies several kernels, producing one feature map each.

The **stride** is the step by which the kernel moves; **padding** adds a border of zeros so the output keeps a chosen size.

2.2 Pooling

A **pooling layer** reduces the spatial size of a feature map by summarizing each small region with a single value. **Max pooling** takes the maximum of each region; **average pooling** takes the mean. Pooling lowers the resolution and computation and makes the representation less sensitive to small shifts.

2.3 Receptive fields

The **receptive field** of a unit is the region of the input that influences its value. Units in early layers see a small patch; stacking convolutional and pooling layers enlarges the receptive field, so deeper units respond to larger and more complex parts of the image. This is **why spatial structure matters**: the network builds from local patterns up to whole-image features.

A CNN for image classification stacks convolutional and pooling layers to extract features, then ends with fully connected layers that map those features to class scores. Its behavior is interpreted by examining which patterns the feature maps respond to.

3 Transfer learning

Transfer learning reuses a model already trained on a large dataset instead of training from scratch. **Pretrained models** such as **ResNet** and **EfficientNet**, trained on large image collections, provide convolutional layers that already extract general visual features.

Two ways of reusing a pretrained model:

- **Feature extraction**: freeze the pretrained layers and train only a new final classifier on top. The pretrained weights are unchanged. Appropriate when the new dataset is **small** or close to the original, and when training must be cheap.
- **Fine-tuning**: continue training some or all of the pretrained layers on the new dataset, usually with a small learning rate. Appropriate when the new dataset is **larger** or differs more from the original, so the features themselves should adapt.

The approach is justified by the size of the available data and its similarity to the pretraining data.

4 Training diagnostics

These are the practitioner skills that Math’s gradient-flow theory prepares but does not practice: reading a training run and fixing it.

4.1 Reading loss and accuracy curves

Plotting the **loss** and **accuracy** on the training set and on a validation set across epochs diagnoses the run:

- **Overfitting:** training loss keeps falling while validation loss rises (or validation accuracy falls). The model memorizes the training data.
- **Underfitting:** both training and validation loss stay high. The model is too simple or undertrained.
- **Vanishing gradients:** the loss barely changes and early layers stop learning, because gradients shrink toward zero as they propagate backward.

4.2 Remediation

- **Learning rate scheduling:** lower the learning rate over training (e.g. step or cosine schedules) so large steps explore early and small steps converge later.
- **Regularization:** counter overfitting with techniques such as weight decay, dropout, or data augmentation.
- **Batch normalization:** normalize each layer’s inputs within a mini-batch. It stabilizes and speeds up training and reduces sensitivity to initialization, which also helps gradient flow.
- **Early stopping:** stop training when validation performance stops improving, keeping the best model and avoiding further overfitting.

Each diagnosis points to a matching remedy: overfitting → regularization or early stopping; underfitting → larger model or longer training; vanishing gradients → batch normalization, better activations, or architectural changes.

5 The Transformer as a building block

The **Transformer** connects Math S4’s attention formulas to the architecture used in GenAI. This section is an architectural walkthrough, not a re-derivation of attention.

5.1 Encoder and decoder stacks

A Transformer is built from stacked identical **blocks**. An **encoder** block combines a **multi-head self-attention** sublayer — the attention mechanism of Math S4, run several times in parallel as separate **heads** — with a position-wise **feedforward** sublayer. Each sublayer is wrapped with a **residual connection** and **layer normalization**. A **decoder** block adds a second attention sublayer that attends to the encoder’s output, and its self-attention is **masked** so each position attends only to earlier positions. The encoder and decoder each stack several such blocks.

5.2 Positional encoding

Attention itself is order-independent: it treats the input as a set. **Positional encoding** adds, to each input embedding, a vector that depends on the position of the token, so the model can use word order. Without it the Transformer could not distinguish different orderings of the same tokens.

5.3 The full forward path

The forward path of an encoder–decoder Transformer is:

1. embed the input tokens and add positional encodings;
2. pass them through the stack of encoder blocks (self-attention then feedforward, each with residual and normalization);
3. in the decoder, embed the output tokens, apply masked self-attention, attend to the encoder output, and apply the feedforward sublayer, through the decoder stack;
4. project the final decoder representations to scores over the vocabulary.

This shows how the attention mechanism learned in Math S4 is assembled into the full architecture.

6 Model comparison methodology

Neural networks are not always the right choice. They are compared against classical machine learning models — in particular the tree-based models of DAT42-1 — along three axes:

- **Latency:** tree-based models are typically faster and cheaper to run at inference; large neural networks can be slower.
- **Interpretability:** tree-based models are easier to inspect and explain; neural networks are largely opaque.
- **Data requirements:** neural networks usually need large datasets to outperform; on small or tabular datasets tree-based models often match or beat them.

Neural networks tend to win on large, high-dimensional, unstructured data (images, text, audio) where they learn features directly; tree-based models tend to win on small or medium tabular data where speed and interpretability matter. The choice on a given problem is justified by weighing these three factors.

7 End-to-end neural network project

The course culminates in a complete project. The student must:

1. **train** a neural network on a real dataset;
2. **evaluate** it with appropriate metrics on held-out data;
3. produce a **model card** — a short document recording the model’s purpose, data, architecture, training setup, evaluation results, and limitations;
4. **defend the architecture choice** to a technical audience, justifying it against alternatives using the comparison methodology above.